

Canonical source: 00-CHARTER/GUIDES/RECEIPTS_AND_AUDIT.md

Æ Orange AI Computer Receipts And Audit

A receipt is evidence that a named action reached a named result under a named contract. It is not a decorative log line, a model-authored success claim, or a promise that unrelated paths are healthy.

What A Receipt Must Answer

- What action was requested?
- Which scope, risk, and approval applied?
- Which executor and runtime path acted?
- Was execution attempted or only planned?
- What evidence proves the result?
- Which hashes, sequence, and predecessor preserve lineage?
- Which blockers or caveats remain?
- What exact claim may an auditor make from it?

Operational input uses `orange.order.v1` ; operational output uses `orange.report.v1` . Models may propose content, but deterministic code owns schema validation, execution attestation, evidence checks, and receipt writing.

Evidence Precedence

Use the freshest evidence for the exact path:

1. semantic live probe;
2. current hash-chained receipt;
3. current executable test;
4. current source or configuration;
5. runtime authority and public manuals;
6. historical plans and chat claims.

A newer narrow failure can supersede an older broad green result for that one path. A newer narrow success cannot promote paths it did not exercise.

Chain And Storage

Public proof artifacts live under `10-RECEIPTS` . Machine-local mutable evidence may also live under `%USERPROFILE%\OrangeBox-Data\orange5` . Receipt chains use SHA-256 lineage so mutation, missing predecessors, sequence breaks, and mixed chains can be detected.

Hash integrity proves that recorded bytes remain linked. It does not prove that the original observation was correct. Strong audit combines chain validation with schema validation, executor identity, runtime evidence, and independent reproduction.

Audit A Claim

1. Rewrite the claim as one falsifiable sentence.
2. Identify the exact runtime crossing and acceptance conditions.
3. Locate the newest receipt for that crossing.
4. Verify the receipt exists and validates against its schema and chain.
5. Inspect raw evidence rather than the summary alone.
6. Reproduce with the named command when risk and environment permit.
7. Check for a later contradictory receipt or live probe.
8. State the boundary: what the evidence proves and what it does not.

For native or visual work, link pixels to process, run, model or gateway request, and receipt identity. A screenshot without that linkage proves only the pixels.

Receipt States

State	Meaning
PROVEN	named runtime path satisfied its acceptance contract
CONFIGURED	implementation or configuration exists; runtime proof is pending
DEGRADED	path is callable with a named limitation
BLOCKED	required dependency, authority, or gate is missing
FUTURE	research or candidate work only

Process start, HTTP 200, file existence, model installation, unit-test success, and agent `done` are observations. None is universal completion evidence.

Public Citation Form

Every public metric should name:

```
claim + workload + denominator + environment + timestamp  
+ receipt path + reproduction command + limitations
```

Avoid naked ratios and adjectives. Confidence comes from a reader being able to inspect the same evidence and reach the same bounded conclusion.

Verify

```
cd C:\AtomEons\Orange5  
bun run verify:json
```

Use focused proof commands from [Proof and Benchmarks](#) for the claimed subsystem. The full verifier is a broad snapshot, not a substitute for a fresh exact-path receipt.

Related Guides

- [Proof and Benchmarks](#)
- [Memory and Learning](#)
- [Troubleshooting and Recovery](#)
- [Skeptic's Field Guide](#)